



2021 真题解析

一、单选选择题

1	2	3	4	5	6	7	8	9	10
A	C	D	A	C	C	B	B	A	C

二、填空题

11. 0x1200 0x53 0xa4

12. %rdi %rsi %rdx %rax 用户栈

13. 试图访问的指令不合法/访问的页面不存在 私有的写时复制

14. 3 11 38

15. 7 34 0.5 32

16. 内存泄漏 垃圾回收

17. gcc -S hello.c (-o hello.s)

三、分析题

18 (1) 为表 1 中汇编代码写出每条指令的注释

myproc:

指针 x 存放在寄存器%rdi 变量 n 存放在寄存器%esi 变量 k 存放在寄存器%edx

movl \$0,%ecx 设置变量 i=0

movl \$0,%r8d 设置变量 m=0

movl \$0,%eax 设置变量 val=0

jmp .L2 直接跳转到标号 L2 处执行.L3

movslq %ecx,%r9 将变量 i 进行符号扩展

addl (%rdi,%r9,4),%eax 将 x[i] 的值累加在变量 val 中



addl \$1,%r8d m=m+1

addl \$edx,\$ecx i=i+k

.L2

cmpl %esi,\$ecx 比较 i 和 n (这里注意顺序)

j1 .L3 如果小于, 跳转到标号 L3 处执行

testl %r8d,%r8d 测试 m 是整数负数还是零

jle .L1 如果是负数或者零, 跳转到标号 L1 处执行

cld 被除数 val 做符号扩展

idivl %r8d val=val/m 商在寄存器 eax 里

.L1

.ret 函数返回 val 的值

(2) 将表 1 中 C 语言函数补充完整

① =i+k ② <n ③ val+x[i] ④ m+1 ⑤ m>0

19 (1) 组合逻辑包含 ALU(算术逻辑单元)、控制逻辑、指令内存、(寄存器文件的读操作、数据内存的读操作); 状态单元包括 PC 寄存器、条件码寄存器、数据内存、寄存器文件。

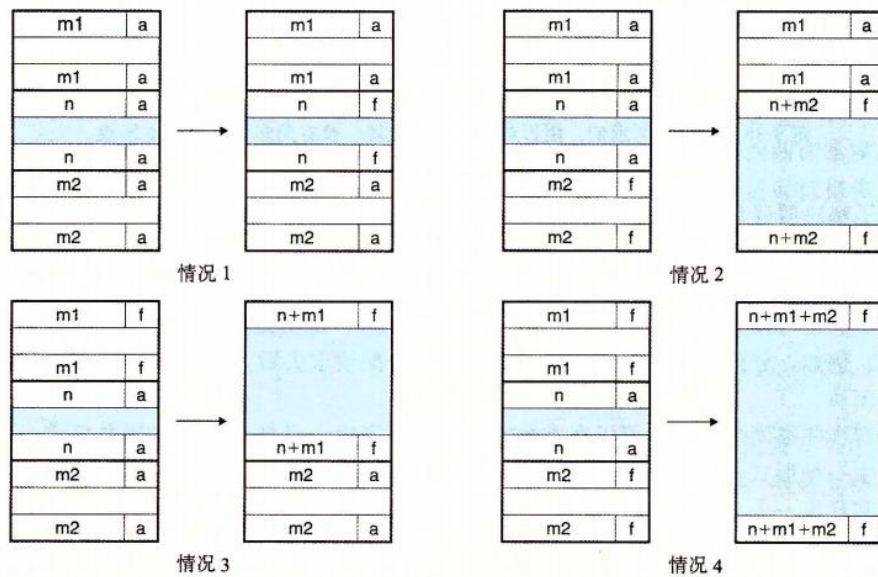
时刻①: CC:100 PC:0x014 rbx:0x100 组合逻辑是当前指令 addq 的结果

时刻②: CC:000 PC:0x016 rbx:0x300

(2) 基本设计思路: 将一个过程划分为几个独立的阶段, 移动目标依次顺序通过每一个阶段。这样在任何时刻, 都会有多个目标正在被处理。划分的阶段数过多时, 由寄存器更新引起的延迟会成为严重的性能限制因素。



四、综合设计题



在情况 1 中，两个邻接的块都是已分配的，因此不可能进行合并。所以当前块的状态只是简单地从已分配变成空闲。在情况 2 中，当前块与后面的块合并。用当前块和后面块的大小的和来更新当前块的头部和后面块的脚部。在情况 3 中，前面的块和当前块合并。用两个块大小的和来更新前面块的头部和当前块的脚部。在情况 4 中，要合并所有的三个块形成一个单独的空闲块，用三个块大小的和来更新前面块的头部和后面块的脚部。C 语言函数 `mm_free_coalesce` 代码见书 601 面。



```
1 void mm_free(void *bp)
2 {
3     size_t size = GET_SIZE(HDRP(bp));
4
5     PUT(HDRP(bp), PACK(size, 0));
6     PUT(FTRP(bp), PACK(size, 0));
7     coalesce(bp);
8 }
9
10 static void *coalesce(void *bp)
11 {
12     size_t prev_alloc = GET_ALLOC(FTRP(PREV_BLKBP(bp)));
13     size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKBP(bp)));
14     size_t size = GET_SIZE(HDRP(bp));
15
16     if (prev_alloc && next_alloc) { /* Case 1 */
17         return bp;
18     }
19
20     else if (prev_alloc && !next_alloc) { /* Case 2 */
21         size += GET_SIZE(HDRP(NEXT_BLKBP(bp)));
22         PUT(HDRP(bp), PACK(size, 0));
23         PUT(FTRP(bp), PACK(size, 0));
24     }
25
26     else if (!prev_alloc && next_alloc) { /* Case 3 */
27         size += GET_SIZE(HDRP(PREV_BLKBP(bp)));
28         PUT(FTRP(bp), PACK(size, 0));
29         PUT(HDRP(PREV_BLKBP(bp)), PACK(size, 0));
30         bp = PREV_BLKBP(bp);
31     }
32
33     else { /* Case 4 */
34         size += GET_SIZE(HDRP(PREV_BLKBP(bp))) +
35             GET_SIZE(FTRP(NEXT_BLKBP(bp)));
36         PUT(HDRP(PREV_BLKBP(bp)), PACK(size, 0));
37         PUT(FTRP(NEXT_BLKBP(bp)), PACK(size, 0));
38         bp = PREV_BLKBP(bp);
39     }
40     return bp;
41 }
```



五、选择题

1	2	3	4	5	6	7	8	9	10
B	C	C/D	D	B	A	A	C	D	A

六、综合应用题

(1)、

目的网络	子网掩码	下一跳 IP 地址	接口
0.0.0.0	0.0.0.0	210.1.1.41	L1
192.168.12.0	255.255.255.0	192.168.1.2	L0
210.1.2.3	255.255.255.255	/	E1
192.168.12.192	255.255.255.192	/	E2

(2)、H2 发送给 H1 的数据报 P 的地址 1, 地址 2, 地址 3 分别是: 00-11-22-33-44-EE, 00-11-22-33-44-DD, 00-11-22-33-44-CC。

AP 转发报文 P 的目的 MAC 地址是: 00-11-22-33-44-CC, 源 MAC 地址是: 00-11-22-33-44-DD。

交换表项:

<00-11-22-33-44-DD, 3>

(3)、

公用 IP 地址	公用端口号	私网 IP 地址	私网端口号
210.1.1.42	5001	192.168.12.196	80

(4)、3 片

总长度	片偏移
596B	0
596B	72
348B	144

七、单选选择题

1	2	3	4	5
A	C	B	C	B

八、问答题

根据题意, 需要遍历完整个数组才能找到所有重复元素。题目要求尽可能快, 可以以空间换时间。

设立一个长度为 n 的数组 $b[n]$ 初始化全 0

遍历已存在的数组, 假设为 $a[n]$

读取 $a[i]$ 其为取值范围 $0 \sim n-1$ 整数 $b[a[i]]++$

遍历完 a 数组后, 遍历输出 b 数组, 若当前数组元素非 0 非 1, 则其重复出现, 此时输出 b 数组下标 (不是输出 b 数组的元素!)

时间 $O(n)$ 空间 $O(n)$



九、算法设计题

```
struct ListNode
{
    int data;
    ListNode *left;
    ListNode *right;
};
```

```
ListNode* search(ListNode* cur, ListNode *p)
{
    ListNode *max = NULL;
    num=p->data;

    while(cur != NULL)
    {
        if(cur->data < num)
        {
            max = cur;
            cur = cur->right;
        }
        else cur = cur->left;
    }
    return max;
}
```



详细解析部分

一、单项选择题

1. 本题主要考察第三章汇编的基础概念。一条指令的执行过程，CPU 可以不进行访存也可以多次进行访存，故 A 错误。RIP 寄存器存放的是 PC 的值也就是下一条指令的地址，因此 CPU 总是执行其所指向的指令，call 和 ret 的函数调用和跳转也是通过它实现的。
2. 本题主要考察第二章数据类型的转换。在整数之间进行转换，二进制的位级是始终不会改变的，因为补码和无符号数不过是对相同位的不同解释，但整数转换会根据类型长度的不同进行截断和扩展。本题 C 选项在整数转换时对二进制进行重新编码，故错误。
3. 本题考察的是第二章一个较细的知识点，在有符号数中，若 $x=TMin$ ，则 $-x=TMin$ ，即有符号数的最小值的相反数还是它本身，故 D 选项错误。
4. 本题考察第四章条件码以及条件跳转。JE 代表相等时跳转，对应需要查看 ZF（零标志）条件码，判断运算结果是否为零。条件码讲解参见课本 P135，有条件跳转指令参见课本 P139。
5. 本题考察 X86 汇编指令以及字节码序列相关知识点，略有超纲。在立即数与寄存器做加法运算时，X86 中的 add 指令对应两种字节码 81 和 83。其中 81 代表立即数是 unsigned 数，需要将立即数进行零扩展至与寄存器的相同字节，83 代表立即数是 signed 数，不需要做任何扩展。又因在 X86 中是小端法存储，故本题应选 C。

注：Y86 的立即数只能是 8 字节，X86 的立即数是变长的。

6. 本题考察第六章高速缓存相关知识点。我们通常用不命中率来衡量系统的性能，因为 cache 不命中的会访问内存，每次不命中会有较大的时间开销。

- 在命中和不命中之间差距巨大
 - 如果只有 L1 和主存，那么可以差 100 倍

- 你会相信 99% 命中率要比 97% 好两倍？

- 思考：
 - 缓存命中时间为 1 个周期
 - 不命中处罚要 100 个周期

- 平均访问时间：

97% 命中率：1 周期 + 0.03 × 100 周期 = 4 周期
99% 命中率：1 周期 + 0.01 × 100 周期 = 2 周期

- 这就是为什么用“不命中率”而不是“命中率”

1) Improve the average access time
要提高平均访问时间，必须提高命中率！
 α HIT RATIO: Fraction of refs found in CA
(1- α) MISS RATIO: Remaining references.
$$t_{avg} = \alpha t_h + (1-\alpha)(t_c + t_m) = t_h + (1-\alpha)t_m$$

2) Transparency (compatibility, programming ease
Cache 对程序员来说是透明的，以方便编程



7. 本题考察第七章链接中的强弱符号。其规则有三：1) 不允许有多个同名的强符号。2) 如果有一个强符号和多个弱符号同名，那么选择强符号。3) 如果有多个弱符号同名，那么从这些弱符号中任意选择一个。故本题选 B。
8. 本题考察第九章虚拟内存相关内容，CR3 控制寄存器指向第一级页表的起始位置，又页表是存放在 cache 或者物理内存中的，故该题选择 B 物理地址。
9. 虚拟页的大小是固定的，在 Intel Core i7 中通常为 4KB，故每一个虚拟页其起始地址都是 4KB 的倍数。故本题选 A。
10. 本题考察第八章信号的相关知识点。如果一个进程有一个类型为 k 的待处理信号，那么任何接下来发送到这个进程的类型为 k 的信号都不会排队等待，并且内核默认阻塞任何当前处理程序正在处理信号类型的待处理信号。故 C 选项中，新到达的信号 k 会变被阻塞变成待处理信号但不会被计数。故 C 选项是错误的。

二、填空题

11. 题主要考察第三章整数计数器的理解。将立即数 $0x1200$ ，存入 `%ebx` 中，并将 `%rbx` 的高四字节自动复制为零，故寄存器 `rbx` 的值为 $0x1200$ （前面的零可以省略）；因为数据在内存中是小端法存储的，故会将地址放入寄存器的低字节，所以之后会从地址 $0x1202$ 访存并将 $0x55\ 54\ 53\ 52$ 四个字节写入 `%eax` 中 `%ah` 代表寄存器 `%ax` 高一字节，故此时 `%ah` 的值为 $0x53$ ；执行第三条指令后，会将 $0x53525150$ 与 $0x55545352$ 相加，并将结果存入 `%eax` 中，此时 `%ah` 的值为 $0xa4$ 。
12. 前三个参数寄存器及其顺序为 `%rdi`, `%rsi`, `%rdx`，函数的返回值保存在寄存器 `%rax` 中。如果函数的参数个数超过六个，则超过部分通过用户栈传递。
13. 解答略，参考课本 p581-p584 页。
14. 通过块大小为 8 可知块内偏移为 3 位，由 $C = S * E * B$ 可知 $S = 2^{11}$ ，故 $s = 11$ ；又由题意可知物理地址为 52 位，所以 $tag = 52 - 11 - 3 = 38$ ；



15. 页面大小为 2KB，故 $VPO=PP0=11$ ， $VPN = 42-11 = 31$ ，并且 $VPN2=VPN3=VPN4=8$ ， $VPN1 = 31-3*8=7$ ； $PPN = 45-11 = 34$ 。页表表项长度为 8 字节，故每页有 $2KB/8B = 0.25K$ 个页表条目。三级页表中每个页表条目指向一个四级页表，每个四级页表有 0.25K 个页表条目，四级页表中的每个页表条目映射一个 2KB 的页，故每个三级页表条目映射 $0.25KB*2KB = 0.5MB$ 的内存区域，一级页表的计算同理，不再赘述。（如果还没有理解，参考一下第九章视频，我有详细讲解）

16. C 语言中使用 `malloc()` 动态申请内存后没有显式的释放会造成内存泄漏，具有垃圾回收机制的编程语言会自动的回收垃圾，故不存在此问题。

17. 2018 期末卷原题，详细解答略。

三、分析题

1. 本题是一道较为综合的汇编代码考察题，涉及了数据传送、算数逻辑运算、跳转等指令和条件寄存器、过程调用等汇编内容。写这类题目的关键在于清楚寄存器内存放值的含义，并根据汇编代码手动更新寄存器的值。汇编代码的解析如第一部分所示。需要注意的几个地方在于参数传递的顺序（`rdi`、`rsi`、`rdx`...）、局部变量赋值的顺序以及指令操作数顺序的含义（例如 `cmp` 指令）。第二问，此类从汇编代码反推 C 语言的题目，需要熟练掌握什么样的 C 语言会生成什么样的汇编代码，什么样的 C 语言结构会产生什么样的汇编结构。如果对汇编不熟悉的话一定要敢于猜测，因为这类题目 C 代码一般不复杂。第一个空填写的是循环结束后循环变量 `i` 的变化，答案从指令 `addl %edx, %ecx` 获得。第二个空填写循环结束的条件，答案从 `cmpl %esi, %ecx` 和 `jle .L3` 共同获得。第三四个空填写循环内的语句，根据变量所在寄存器的位置以及循环汇编的结构从 `.L3` 后的三条语句获得。最后一个空填写条件分支的判断语句，从 `testl %r8d, %r8d` 和 `jle .L1` 获得。需要注意的是 `test` 的两种基本用法，一是两个相同寄存器的与用来测试寄存器内的数是整数、负数还是零；二是和常数与，来判断寄存器的某些位是否为 1。`jle` 表示小于等于，也就是测试结果为负数或零时跳转到 `ret` 执行，因此条件分支应该为 `m>0`。

2. 本题考察 Y86 的顺序实现（SEQ）的相关内容。第一问重点关注 SEQ 中组合逻辑与状态单元的区别。在周期 3 结束前一刻，组合逻辑已经完成功能，而状态单元需要等到时钟上升沿到来后，即周期 4 开始的后一刻，完成状态的更新，例如题目中间到的 `CC`、`PC`、`rbx`。这部分内容在书中的 276 面（注意书中 `CC` 初始状态 100 那里打印错误）。第二问较为基础，考察流水线设计的思想与流水线产生的问题，此问没有标准答案，意思与给出的答案相近即可。

四、综合设计题



本题考察隐式空闲链表的释放与合并操作，两问分别考察合并的原理与具体的实现代码。第一问是第二问的铺垫，第二问具有一定的难度。第一问只要记得书上 596 面图，即合并的四种情况（最好能画下来），并能够用文字表述即可。第二问是书中 9.9.12 综合：实现一个简单的分配器的内容。题目当中给出了和书中相同的宏和宏函数，理解这些宏的作用是关键。书中对宏的作用有详细阐述。对于头部和脚部而言我们知道它是由块大小 size 和已分配位 alloc 共同决定的，完成这两部分组装成完整头部或脚部的是宏 PACK(size, alloc)，同样也有提取 size 和 alloc 的操作宏 GET_SIZE 和 GET_ALLOC。我们能够操作的指针有两类，一类是指向字（4 字节）的用 p 来表示（就是指向头部和脚部），一类是指向块的用 bp 来表示。读取头部（脚部）值或修改的宏是 GET(p) 和 PUT(p, val)；从 bp 转化为 p（读取块的头部脚部）的宏是 HDRP(头部) 和 FTRP(脚部)；获取前一块 bp 和后一块 bp 的宏是 NEXT_BLK 和 PREV_BLK。理解宏定义并结合四种情况的图，代码写起来较为容易，参考书中 601 面的代码及相关注释。

五、选择题

1. 本题考查的是两个参考模型的比较，请同学们熟悉 OSI 和 TCP/IP 参考模型。OSI 有七层分别是：应用层，表示层，会话层，传输层，网络层，数据链路层，物理层。TCP/IP 有 4 层分别是：应用层，传输层，网际层，网络访问层。具体对应关系参考下图。因此 OSI 模型中实现表示转换功能的自然是表示层，而 TCP/IP 中实现该功能的是应用层，答案选 B。



2. 该题与平时做的 mooc 题目来比相对偏简单，所以基本上只要会了 mooc 上题目，这题很容易。答案是 A。另一种相对简单的做法是，由于是分组交换，所以网络的带宽等价于 H1 和 H2 两人各自分得一半，互相独立。因此 H1 的独立带宽是：从 100M 到 500M 到 500M 到 100M，而 H2 的独立带宽是从 10M 到 500M 到 500M 到 100M。所以计算 H1 和 H2 的时间，公式 $t = \text{文件大小} / \text{最低带宽} + \text{最后一个分组在中间路由器的转发时间}$ 。（注意 1B=8bit）



3. 典型考察访问解析域名+TCP 连接的过程。两种方法迭代与递归（这两种方法的时间是一样的）。首先此处本地域名服务器没有缓存，所第一步先访问本地域名服务器，然后没有解析 IP，进一步通过路由器访问了 xyz.com 域名服务器，解析得到 IP 地址，然后建立 TCP 连接和请求 index.html 需要 2 个 RTT。所以总共需要 4 个 RTT，答案是 D。示意图在这里很重要。默认情况下，我们一般是本地，根，com, xyz.com 才能获得 IP 地址，这里示意图告诉我们可以直接访问 xyz.com 域名服务器。

另一种答案是 C：本地域名服务器不算时间，题目说的是：局域网内主机访问 internet 上服务器的往返时间是 RTT。

4. 答案 D。邮件服务器之间只能是 SMTP 协议。而发送端到发送服务器的协议可以是 smtp, http, 接收服务器到接收端可以是：pop3, http, IMAP
5. 这题与传统的题目有点区别的是，3 比特编号，而接收端的窗口是 3，则发送端的窗口只能是 5。因此先计算周期 $T=16\text{ms}$ ，发送一个帧长的时间是 0.8ms ，最多发送 5 个，因此 $0.8 \times 5 / 16 = 25\%$
6. 翻倍即可。
7. 此题有一定的迷惑性。乙发送的 $\text{ack_seq}=1000$ ，表示的是前面的数据我都收到了，我确认了，请发给我以 1000 开头的数据帧。结构甲发的是 1001 和 1301 开头。说明 1000 开头的帧，携带 1 字节的数据丢了。那么乙应该重新发送确认 $\text{ack_seq}=1000$ 的确认帧。这个题目很容易错选 1501。（因为 1000 和 1001 之间，很容易让人蒙了，以为 $\text{ack_seq}=1000$ ，表示的是 1000 的数据已经收到了，请发送 1001 为开始的帧）
8. C
9. D（如果有 1，3，4，也可以选择。因为 VLAN 是可以通过交换机实现的）
10. 码元速率 $= 2 \times W$ (奈奎斯特定理)，其实码元的波特率是奈奎斯特规定的 $B = 2W = 4\text{M}$ ，你可以通过奈奎斯特求速率： $v_1 = B \log_2(16) = 16\text{M}$ 。同时香农定律，在有噪声的环境下的 $v_2 = W \log_2(1+s/n)$ 约等于 20M 。其实最高速率 $v = \min(v_1, v_2)$ 。这题的意思是，通过香农定律求得数据传输率为 20M ，然后一个码元编码 4 字节信息，因此 $B = 20\text{M} / 4 = 5\text{M}$ 。

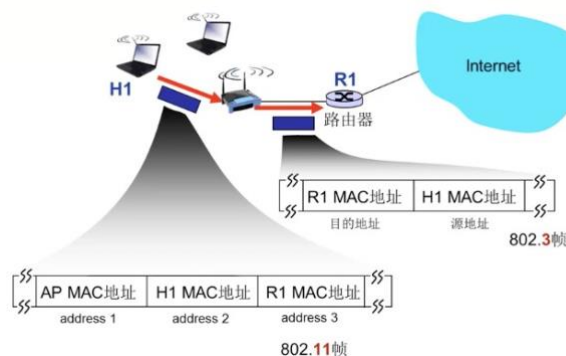
六、综合应用题

- (1) R2 需要路由的网络有：R1，DNS 服务器，R3，web 服务器，H1，H2

首先，不同路由接口肯定是不能聚合在一起的。因此 Internet 独立，DNS 服务器独立，R3 上的两个客户端可以聚合，然后再看看 web 服务器，H1，H2 聚合之后是否有冲突。注意：路由器 R1 与 路由器 R2 的 L1 端口相连的端口 ip 地址可以间接求出，因为网络号 30，那么主机号就 2 位，总共 2 个可用地址（去掉全 0，全 1），故留给这里的 ip 地址只有一个。

(2) 需要弄清楚 802.11 帧的格式（重点，几乎必考）。交换机泛洪之后，H1 是会产生回复，因此 H1 的表项不在交换机中。

去往 AP (To AP)	来自 AP (From AP)	地址 1	地址 2	地址 3
0	1	目的地址	AP 地址	源地址
1	0	AP地址	源地址	目的地址



(3) 答案很直接。私有 ip 的端口必须是 80（因为服务是在这里），其他随便编。

(4) 注意分片的数据段必须是 8 的倍数。因此第一个段 600-20，不能整除 8，因此取最近的 576。所以第一个段 576+20，第二个段 576+20，第三个段 328+20。总体有效数据是 1480B。（有些同学有疑问，为什么最后一个段可以不用整除 8，因为最后一个段的偏移量用不到，最后一个段的偏移量只跟前面段的数据长度有关，分片长度为什么要是 8 的倍数的原因是：IP 报的偏移量*8=实际的字节偏移。）

七、单项选择题

3、 $n(n-1)/2 \geq 22$ 8

4、就是一棵满二叉树

八、问答题

九、算法设计题

